

The Mixed-Kernel H-LVGP (`heter_lvgp_torch_mixed`): a adaptation of botorch’s `MixedSingleTaskGP` kernel algebra to the latent-variable GP

lvgp_modeling_study — implementation note

2026-07-31

1 Motivation (what the 30-seed study showed)

Across the study’s replicated benchmarks the Categorical GP (botorch `MixedSingleTaskGP`) beat the latent-variable H-LVGP in most configurations. Two structural mechanisms were isolated:

1. **The offset tax.** The LVGP has a single product kernel and one constant mean, so a pure between-level offset can only be encoded by pushing latent vectors apart — which decays the correlation for *everything*, offsets and shape alike (demonstrated by the Griewank $b(\ell)$ redesign, which flipped every budget to the Categorical GP).
2. **The additive channel.** botorch’s mixed kernel contains an *additive* categorical component, equivalent to a per-level random-effect offset $h(\ell)$ carried in the covariance: $f(x, \ell) = g(x) + h(\ell) + \text{interaction}$. This absorbs offsets at the cost of a single outputscale.

The mixed-kernel H-LVGP grafts botorch’s kernel algebra onto the LVGP: the latent kernel takes the categorical slot, so the model keeps the interpretable latent embedding *and* gains the offset channel. Notably, its level random effects are correlated through latent distance — similar levels share offsets — which even the Categorical GP cannot express (its offsets are exchangeable).

2 The botorch reference implementation

botorch v0.14.0, `models/gp_regression_mixed.py` (L109–152) builds, for continuous dims x and categorical dims c :

$$K_{\text{sum}} = \sigma_s^2 \left(k_{\text{RBF}}^{(1)}(x, x') + \sigma_c^2 k_{\text{cat}}^{(1)}(c, c') \right) \quad (\text{L124–138})$$

$$K_{\text{prod}} = \sigma_p^2 k_{\text{RBF}}^{(2)}(x, x') \cdot k_{\text{cat}}^{(2)}(c, c') \quad (\text{L139–151})$$

$$K = K_{\text{sum}} + K_{\text{prod}} \quad (\text{no global scale}) \quad (\text{L152})$$

with the following exact ingredients:

- **Independent instances** for the sum and product terms (class docstring L41–43: “we fit different lengthscales for the kernels in the sum and product terms”).
- **Continuous kernels:** ARD RBF from `get_covar_module_with_dim_scaled_prior` (`models/utils/gpytorch_modules.py` L100–133): lengthscale prior $\ell_i \sim \text{LogNormal}(\sqrt{2} + \frac{1}{2} \ln d, \sqrt{3})$ (the Hvarfner-2024 dimension-scaled prior, assuming unit-cube inputs), hard floor $\ell_i \geq 0.025$ via `GreaterThan(0.025, transform=None, initial_value=prior.mode)`.

- **Categorical kernels:** $k_{\text{cat}}(c, c') = \exp(-d_H(c, c')/\ell_c)$ with Hamming distance d_H , per-dim $\ell_c \geq 10^{-6}$, *no prior* (`models/kernels/categorical.py`).
- **Scales:** the inner categorical term of the sum part carries its own `ScaleKernel` (L130); both parts carry outer `ScaleKernels`; none carry priors.
- **Mean:** `ConstantMean`; outcome standardized (`Standardize`); known observation noise via `train_Yvar`.

The Categorical GP’s kernel, precisely

The kernel the study’s Categorical GP uses in the categorical slots is `botorch`’s `CategoricalKernel` (`models/kernels/categorical.py`):

$$k_{\text{cat}}(c, c') = \exp\left(-\frac{d_H(c, c')}{\ell_c}\right), \quad d_H(c, c') = \mathbb{I}[c \neq c'] \quad (1)$$

(averaged over categorical dims when there are several; here there is one). Two properties matter: (i) it is *exchangeable* — d_H gives every pair of distinct levels the same distance, so one lengthscale ℓ_c sets a single common between-level correlation e^{-1/ℓ_c} ; (ii) it is of the form $\exp(-d(\cdot, \cdot))$ for a valid metric d , which is what makes it positive semi-definite.

3 The adaptation: latent kernel in the categorical slot

The LVGP embeds level ℓ as $\mathbf{z}_\ell \in \mathbb{R}^2$ (`lvgp-bayes LVMapping`) and evaluates all kernels on the embedded input $u = [\mathbf{z}, x]$. The mixed-kernel LVGP (`LVGPRMixed`) is

$$\begin{aligned} K((x, \ell), (x', \ell')) &= \sigma_s^2 \left(k_{\text{RBF}}^{(1)}(x, x') + \sigma_z^2 e^{-\|\mathbf{z}_\ell - \mathbf{z}_{\ell'}\|^2 / (2\ell_z^2)} \right) \\ &\quad + \sigma_p^2 k_{\text{RBF}}^{(2)}(x, x') e^{-\|\mathbf{z}_\ell - \mathbf{z}_{\ell'}\|^2 / 2} \end{aligned} \quad (2)$$

i.e. `botorch`’s exact algebra with each `CategoricalKernel` replaced by an RBF over the shared latent embedding:

botorch construct	this adaptation	rationale
$k_{\text{cat}}^{(2)}$ (product slot)	RBF over $\mathbf{z}$, $\ell \equiv 1$ fixed	LVGP gauge: the latent scale lives in $\mathbf{z}$ itself (identical to <code>LVGPR</code> ’s <code>qual_kernel</code>)
$k_{\text{cat}}^{(1)}$ (sum slot, own ℓ)	RBF over $\mathbf{z}$, own learnable ℓ_z	botorch fits independent categorical lengthscales per term
$k_{\text{RBF}}^{(1)}, k_{\text{RBF}}^{(2)}$	two independent ARD RBFs over x , Hvarfner prior, floor 0.025	copied verbatim (unit-cube inputs guaranteed by the wrapper, so the prior’s assumptions hold)
inner scale on $k_{\text{cat}}^{(1)}$ (L130)	inner <code>ScaleKernel</code> on the sum-part latent kernel	structural copy
$K_{\text{sum}} + K_{\text{prod}}$, no global scale	<code>covar_module</code> replaced wholesale after <code>GPR.__init__</code>	GPR’s default global <code>ScaleKernel</code> discarded to match <code>botorch</code>

Where the latent-slot RBF comes from, and the role of its lengthscales

Provenance of the RBF structure. The RBF-over-latents is not our invention — it is the original LVGP correlation of Zhang et al. (2019). There, the level ℓ is mapped to coordinates $\mathbf{z}_\ell \in \mathbb{R}^2$ and the *same Gaussian correlation used for continuous inputs* is applied to those coordinates:

$$r((x, \ell), (x', \ell')) = \exp\left(-\sum_i \theta_i (x_i - x'_i)^2 - \|\mathbf{z}_\ell - \mathbf{z}_{\ell'}\|^2\right), \quad (3)$$

i.e. a product of an ARD Gaussian kernel on x and a *unit-lengthscale* Gaussian (RBF) kernel on $\mathbf{z}$. `lvgp-bayes` implements exactly this: its `qual_kernel` is an `RBFKernel` over the embedded latent columns with lengthscale initialized to 1.0 and gradient frozen (`models/lvgp.py` L187–192). Validity is inherited from the RBF being a kernel on $\mathbb{R}^{d_z}$: any finite set of level embeddings yields a PSD level covariance.

The two kernels are the same family with different metrics. Comparing Eq. 1 and the latent factor of Eq. 3: both are $\exp(-d(\ell, \ell')^p / \ell_{\text{scale}})$ decays; the Categorical GP uses the *fixed, exchangeable* Hamming metric (all distinct pairs equidistant), while the LVGP uses a *learned Euclidean* metric $d(\ell, \ell') = \|\mathbf{z}_\ell - \mathbf{z}_{\ell'}\|$. The Hamming kernel is recovered as the special case in which all latents sit at the vertices of a regular simplex — so replacing k_{cat} by the latent RBF in `botorch`’s algebra is a strict generalization, not a substitution of an unrelated object.

Why the product-slot lengthscale is fixed at 1 (gauge). In the latent RBF a lengthscale is redundant with the scale of the embedding itself: the map $\mathbf{z} \mapsto c\mathbf{z}$, $\ell_z \mapsto c\ell_z$ leaves the kernel invariant, so $(\mathbf{z}, \ell_z)$ are not jointly identifiable. Zhang et al. and `lvgp-bayes` resolve this by fixing $\ell_z \equiv 1$ and letting the latent coordinates carry the scale — a gauge choice, not a modeling restriction. The mixed kernel keeps this gauge in the *product* slot, so $\mathbf{z}$ is identified exactly as in the original LVGP (latent maps remain comparable across models).

Why the sum-slot lengthscale is learnable. `botorch`’s docstring (L41–43) requires *independent* categorical lengthscales for the sum and product terms — that freedom is what lets the additive (offset) channel and the multiplicative (shape) channel decay at different rates across levels. With a *shared* $\mathbf{z}$ this freedom cannot be absorbed into the embedding (the product slot already fixes $\mathbf{z}$ ’s scale), so the sum slot must carry its own distance scale ℓ_z : in Eq. 2 the offset channel correlates levels as $\exp(-\|\Delta\mathbf{z}\|^2 / (2\ell_z^2))$. Interpretation: ℓ_z measures how much broader offset-sharing is than shape-sharing, in latent-distance units (fitted example, camel $n=25$: $\ell_z \approx 1.7$ — offsets shared across level clusters whose shapes are already nearly independent).

Everything else is inherited unchanged: the latent mapping and its priors (raw latents $\sim \mathcal{N}(0, \sqrt{L})$, precision $\sim \text{ExpGamma}$), constant mean with $\mathcal{N}(0, 1)$ prior, internal y -standardization (= `botorch Standardize`), and the study wrapper’s `FixedNoiseGaussianLikelihood` carrying the replicate variances (= `botorch train_Yvar`), plus the MATLAB-parity pooled aleatoric polynomial over $[x, \mathbf{z}]$ for noise-surface recovery.

The new kernel vs. the original LVGP kernel

Writing both models with their scales makes the relationship exact. The **original LVGP** (Zhang et al. 2019; LVGPR) is a *single fully-multiplicative* channel,

$$K_{\text{LVGP}} = \underbrace{\sigma^2 e^{-\sum_j \phi_j (x_j - x'_j)^2}}_{\text{shape in } x} \underbrace{e^{-\|\mathbf{z}_\ell - \mathbf{z}_{\ell'}\|^2}}_{\text{level similarity}}, \quad (4)$$

while the **mixed kernel** (Eq. 2) is a sum of three channels,

$$K_{\text{mix}} = \underbrace{\sigma_s^2 k_{\text{RBF}}^{(1)}(x, x')}_{\text{(a) shared trend}} + \underbrace{\sigma_s^2 \sigma_z^2 e^{-\|\Delta\mathbf{z}\|^2 / (2\ell_z^2)}}_{\text{(b) level offsets}} + \underbrace{\sigma_p^2 k_{\text{RBF}}^{(2)}(x, x') e^{-\|\Delta\mathbf{z}\|^2 / 2}}_{\text{(c) = the original LVGP kernel}}.$$

Why and how channels (a) and (b) were added. They were not designed separately — both fall out of botorch’s single *sum part* when it is distributed:

$$K_{\text{sum}} = \sigma_s^2 (k_{\text{RBF}}^{(1)}(x, x') + \sigma_z^2 k_{\text{lat}}(\mathbf{z}_\ell, \mathbf{z}_{\ell'})) = \underbrace{\sigma_s^2 k_{\text{RBF}}^{(1)}}_{(a)} + \underbrace{\sigma_s^2 \sigma_z^2 k_{\text{lat}}}_{(b)}.$$

The statistical reading uses the additivity property of GPs: a sum of kernels is the covariance of a sum of *independent* functions, $f \sim \mathcal{GP}(0, K_a + K_b + K_c) \iff f = f_a + f_b + f_c$. Hence:

- **Channel (a) — why:** a kernel that depends on x only generates a component $f_a(x)$ that is *identical for every level* — the common backbone all categories ride on. The study’s problems (and most physical systems with a categorical design choice) share most of their response across levels; in the original LVGP even this shared part must survive the latent decay $e^{-\|\Delta\mathbf{z}\|^2}$, i.e. it is re-learned per level once latents separate. **How:** an ARD RBF restricted (`active_dims`) to the continuous columns of the embedded input, with its own lengthscales, so the shared backbone may be smoother or rougher than the level-specific shape in channel (c).
- **Channel (b) — why:** a kernel that depends on ℓ only generates a component $f_b(\ell)$ that is *constant in x* — exactly a per-level offset, the random-effects analogue of a per-level mean. This is the surgical answer to the offset tax diagnosed on griewank (adding the $b(\ell)$ shifts flipped every budget to the Categorical GP): offsets now have a dedicated route into the model that costs correlation nothing elsewhere. **How:** the latent-slot RBF evaluated on the *same* embedding $\mathbf{z}$ used by channel (c) — no new level parameters at all — with its own distance scale ℓ_z and the inner outputscale σ_z^2 (botorch L130) setting the offset-to-trend variance ratio; the outer σ_s^2 then weighs the whole additive block against the interaction channel (c). Because the metric is latent rather than Hamming, these offsets are *correlated between similar levels* — the one capability here that even the Categorical GP lacks.

The differences, one by one:

1. **The original kernel survives intact as channel (c)** — same product form, same unit-lengthscale latent factor, its own ARD lengthscales. The original LVGP is exactly the special case $\sigma_s^2 = 0$: the new model *nests* the old one, so it can never be structurally worse, only harder to estimate ($q+4$ extra hyperparameters: one extra ARD set, ℓ_z , and the scale split).
2. **All-or-nothing coupling vs. a functional decomposition.** Under Eq. 4, *every* property of f decays with the same factor $e^{-\|\Delta\mathbf{z}\|^2}$: if two levels sit far apart in latent space they share *nothing* — not a global trend, not a mean shift. The mixed kernel corresponds to $f(x, \ell) = g(x) + h(\ell) + \text{int}(x, \ell)$: channel (a) is shared by *all* levels unconditionally (even latent-infinitely-separated ones), channel (b) carries mean shifts, channel (c) carries level-specific shape. This is precisely why the original pays the “offset tax”: a pure mean shift can only enter Eq. 4 by growing $\|\Delta\mathbf{z}\|$, which simultaneously destroys shape-sharing.
3. **Two latent decay rates instead of one.** The original has a single latent factor; the mixed kernel evaluates the *same* embedding at two rates — $e^{-\|\Delta\mathbf{z}\|^2/2\ell_z^2}$ for offsets, $e^{-\|\Delta\mathbf{z}\|^2/2}$ for shapes — so offset-sharing can extend across level clusters whose shapes are already independent (fitted camel example: $\ell_z \approx 1.7$).
4. **Independent ARD lengthscales per channel.** The original fits one set of ϕ_j ; the mixed kernel fits $k_{\text{RBF}}^{(1)}$ and $k_{\text{RBF}}^{(2)}$ separately (botorch docstring L41–43), so the shared trend may be smoother or rougher than the level-specific shape component.

5. **Consequence for the latent map.** In the original, latent distances conflate offset- and shape-differences; in the mixed kernel offsets are routed through channel (b), so latent distances measure shape similarity more purely — expect tighter, more interpretable level clusters when comparing latent plots across the two models.

4 Documented deviations from botorch, and why they are necessary

4.1 Every new parameter carries a prior

lvgp-bayes’s multistart (`GPR.reset_parameters, models/gpregression.py` L143–149) *samples restart initializations from the registered priors*; a prior-less parameter is silently never restarted, and MAP has no regularization on it. botorch leaves its `ScaleKernel` outputscales and categorical lengthscales prior-less because its own fitter does not need priors. We therefore add, using the package’s own conventions:

- `raw_outputscale` $\sim \mathcal{N}(0, 1)$ on all three `ScaleKernels` (the prior GPR itself uses, L99–101);
- `raw_lengthscales` $\sim \text{MollifiedUniform}(\ln 0.1, \ln 10)$ on the sum-part latent lengthscales ℓ_z (the prior lvgp-bayes puts on every learnable lengthscales, `models/lvgp.py` L208–210).

4.2 Reparameterization of the Hvarfner lengthscales constraint

botorch declares `GreaterThan(0.025, transform=None)` — the raw parameter *is* the lengthscales — because botorch’s fitter enforces constraint bounds inside the optimizer. lvgp-bayes’s `fit_model_scipy` optimizes raw parameters **unbounded** (`optim/mll_scipy.py: bounds=None`). Verified failure mode: L-BFGS walks the raw lengthscales negative and the LogNormal prior’s support check raises `ValueError`, killing every restart silently (the model is then returned at its initial values). Fix: the *identical induced model* — same prior on the lengthscales value, same hard floor — reparameterized per the package convention as

$$\ell = 0.025 + e^{\text{raw}}, \quad \text{raw unconstrained.}$$

Residual edge case, discovered in the 30-seed campaign: a prior draw below 0.025 during a restart makes gpytorch’s bounds check raise, and `fit_model_scipy` calls `reset_parameters` *outside* its per-restart exception handler, so one bad draw ($P \approx 0.2\%$) kills the entire fit (observed once in 210 cells). `LVGPRMixed.reset_parameters` therefore overrides the sampler to clamp such draws just above the floor.

5 Considerations and caveats

- **Identifiability / over-parameterization.** The sum of two scaled parts plus the latent’s own scale freedom is redundant in ways MAP priors must arbitrate (botorch has the same redundancy plus `Standardize`). In practice restarts converge to consistent optima (e.g. camel $n=25$: two of three restarts at the same posterior value).
- **Conditioning.** The additive structure can bring the kernel matrix closer to singular; gpytorch occasionally adds jitter up to 10^{-3} during fitting (observed on friedman $n=80$). Fits succeeded; monitor if reused at larger n .

- **Gauge.** Fixing the *product*-part latent lengthscale at 1 preserves the original LVGP’s identification of $\mathbf{z}$; the sum part’s ℓ_z then measures how much *longer-range* the offset-sharing is than the shape-sharing. Fitted example (camel $n=25$): $\ell_z = 1.66$ — offsets shared more broadly across the latent space than shapes, exactly the intended division of labor.
- **HMC path unsupported.** `LVGPR.to_pyro_random_module` assumes the covariance tree is `ScaleKernel(Product(...))`; the MAP path used by the study does not touch it.
- **Interpretation of the latent map.** $\mathbf{z}$ is still a 2-D level embedding, but distances now encode shape-similarity (product channel) with offset-similarity carried separately (sum channel) — when comparing latent plots against the plain LVGP, expect tighter clusters, since offsets no longer inflate latent distances.

6 Files changed

- `utils/models/lvgp_torch_mixed.py` (**new**) — `LVGPRMixed` (kernel composition of Eq. 2, subclassing the vendored `LVGPR`; vendored package untouched) and `HeterLVGPTorchMixed` (study wrapper: fixed replicate noise + MATLAB-parity aleatoric, inherited).
- `utils/models/lvgp_torch.py` — one-line hook: `_LVGPTorchBase` gained `MODEL_CLS = LVGPR` so subclasses can substitute the GP class; the fit path now constructs `cls.MODEL_CLS`.
- `utils/models/__init__.py` — registry entry `heter_lvgp_torch_mixed` (aux).
- `results_v2/aggregate_seeds.py`, `scripts/export_seed_latex.py` — the new model added to the 30-seed aggregation and both seed-table variants as “H-LVGP (torch-mix)”.

7 Results (30 seeds, fixed design, latex_v2 scope)

Across 30 noise realizations the mixed-kernel model takes the **best MAE in 5 of 7 configurations** (both Branin budgets — including $n=25$, where it beats the MATLAB H-LVGP’s flagship result — and all three Friedman budgets, by 22–25% over the Categorical GP) and the **best interval score in all 7**. The Categorical GP retains camel’s MAE. Coverage sits between 0.86 and 0.99 everywhere. The friedman result is the notable one: the first configuration family where a latent-variable model beats the Categorical GP *decisively and replicably* at every budget — evidence that once the offset channel is provided, the latent geometry is an asset rather than a tax, exactly the mechanism the study’s griewank/camel arcs identified.

Table 1: 30-seed replication (fixed design, fresh noise per seed), latex.v2 scope: mean $\pm$ std over seeds. Bold = best of the four models shown (MAE/IS lower; coverage closest to 0.95).

Config	Model	MAE $\pm$ std	IS	Coverage
Branin $n=25$	Categorical GP	4.529 ± 0.241	38.87	0.946
	H-LVGP	2.520 ± 0.723	37.28	0.996
	H-LVGP (torch)	3.752 ± 0.611	24.61	0.960
	H-LVGP (torch-mix)	2.427 ± 0.510	18.89	0.995
Branin $n=40$	Categorical GP	1.585 ± 0.268	14.86	0.996
	H-LVGP	1.722 ± 0.425	35.61	0.998
	H-LVGP (torch)	1.409 ± 0.195	14.04	0.976
	H-LVGP (torch-mix)	1.183 ± 0.191	11.32	0.987
Camel $n=25$	Categorical GP	0.168 ± 0.009	1.78	0.923
	H-LVGP	0.178 ± 0.012	2.15	0.905
	H-LVGP (torch)	0.231 ± 0.019	2.21	0.917
	H-LVGP (torch-mix)	0.196 ± 0.017	1.58	0.942
Camel $n=40$	Categorical GP	0.114 ± 0.010	1.13	0.918
	H-LVGP	0.143 ± 0.024	1.15	0.953
	H-LVGP (torch)	0.174 ± 0.017	1.45	0.915
	H-LVGP (torch-mix)	0.145 ± 0.019	1.11	0.928
Friedman 5-D $n=40$	Categorical GP	0.883 ± 0.011	7.82	0.873
	H-LVGP	1.242 ± 0.183	14.92	0.712
	H-LVGP (torch)	1.239 ± 0.175	14.19	0.727
	H-LVGP (torch-mix)	0.662 ± 0.026	5.38	0.864
Friedman 5-D $n=60$	Categorical GP	0.483 ± 0.028	3.29	0.963
	H-LVGP	0.552 ± 0.131	3.53	0.910
	H-LVGP (torch)	0.504 ± 0.028	3.47	0.897
	H-LVGP (torch-mix)	0.377 ± 0.026	2.28	0.946
Friedman 5-D $n=80$	Categorical GP	0.317 ± 0.009	2.15	0.962
	H-LVGP	0.345 ± 0.021	2.02	0.973
	H-LVGP (torch)	0.364 ± 0.014	2.35	0.900
	H-LVGP (torch-mix)	0.237 ± 0.009	1.53	0.958